

NSI	LES GRAPHEs PARCOURS D'UN CAMPUS	TD
		Durée : 2h00

Condition de réalisation : A partir d'une représentation graphique d'un graphe

Objectif : Représenter le graphe sous forme d'une liste d'adjacence et/ou d'une matrice d'adjacence. Rédiger ou compléter un programme réalisant le parcours du graphe. Retrouver des chemins à partir d'un dictionnaire des prédécesseurs.

Connaissances associées : Graphe non pondéré, vocabulaire associé aux graphes.

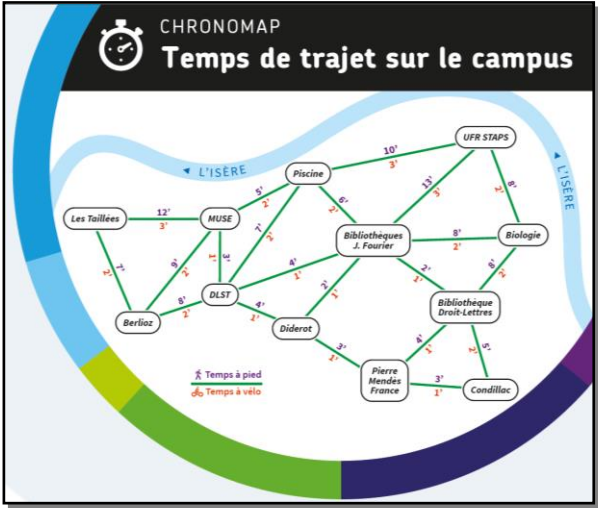
PRESENTATION DU CAMPUS

Le domaine/campus de la ville de Grenoble est localisé dans une ancienne zone marécageuse de deux villes limitrophes à Grenoble (Gières et St Martin d'Hères). Ce campus accueille 35 000 étudiants et 3 000 enseignants.

Sa superficie est de 186 hectares et dispose d'un milieu naturel favorable avec de nombreux arbres (3 000), d'espaces verts et la proximité d'éléments naturels : le fleuve de l'Isère en bordure et le massif de la Chartreuse à l'horizon nord et la chaîne de Belledonne à l'horizon est.

La présence de 3 tramways permettant un accès aisé au campus, ainsi que la présence de pistes cyclables et sentiers incite aux déplacements soit pédestres, soit à vélo.

Un graphe a été élaboré afin de pouvoir de choisir son itinéraire et d'estimer le temps de déplacement soit à vélo ou à pied.

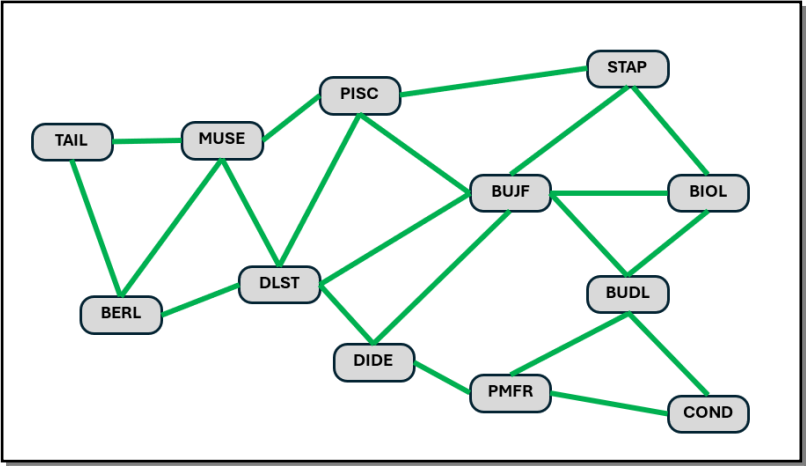


Pour plus d'informations : https://fr.wikipedia.org/wiki/Domaine_universitaire_de_Grenoble

TRAVAIL A REALISER

1/ ÉTUDE SANS PRISE EN COMPTE DES PONDERATIONS

Pour cette 1^{ère} partie, nous vous proposons de ne pas prendre en compte la durée de traversé des arêtes. L'ensemble des lieux sont abrégés avec 4 caractères, nous obtenus alors le graphe suivant :



2/6

Créé le 03/06/2024 9:19 – M.A.J. le 04/06/2024 15:05

- D'une matrice d'adjacence
- D'une liste d'adjacence

[illegible]

```
{ "BUJF": [....., ....., ....., ....., ....., .....],
  "PISC": [....., ....., ....., .....],
  "STAP": [....., ....., .....],
  "BIOL": [....., ....., .....],
  "BUDL": [....., ....., ....., .....],
  "DIDE": [....., ....., .....],
  "DLST": [....., ....., ....., ....., .....],
  "MUSE": [....., ....., ....., .....],
  "COND": [....., .....],
  "PMFR": [....., ....., .....],
  "BERL": [....., ....., .....],
  "TAIL": [....., .....]}
```

1.2/ PARCOURS DE L'ENSEMBLE DU GRAPHE

L'algorithme de parcours complet en largeur du graphe est le suivant :

- Placer le sommet de départ dans une file et une liste de recensement des sommets
- Tant que la file n'est pas vide :
 - Prendre le prochain sommet à visiter dans la file
 - Itérer tous les adjacents du sommet visité :
 - Pour chaque sommet adjacent :
 - Le rajouter dans la file
 - Le rajouter la liste de recensement des sommets
 -

Compléter l'implémentation en Python de l'algorithme de parcours en largeur du graphe :

```
def parcoursLarg(graphe, depart):
    recenses = [ ..... ]
    file = [ ..... ]

    while len(file) != ....:
        sommetEnVisite = file.pop(0)

        # Boucle de récupération des adjacents du sommet visité :
        for adjacent in graphe[sommetEnVisite]:
            if .....not in recenses:
                file. ....
                recenses. ....

    return recenses
```

1.3/ DICTIONNAIRE DES PREDECESSEURS

Nous désirons connaître l'ensemble des chemins à partir d'un sommet de départ vers tous les autres sommets du graphe. Pour ce faire, nous allons utiliser un dictionnaire des prédécesseurs.

Réécrire la fonction `parcoursLarg(graphe, depart)` en remplaçant la liste `recenses` par un dictionnaire des prédécesseurs que nous désignerons : `dicoPredecesseurs`

```
def predecesseurs(graphe, depart):
    dicoPredecesseurs = { ..... }
    file = [ ..... ]

    while len(file) != .....:
        sommetEnVisite = file.pop(0)
        # Boucle de récupération des adjacents du sommet visité :
        for adjacent in graphe[sommetEnVisite]:
            if ..... not in dicoPredecesseurs:
                file. ....
                dicoPredecesseurs[ ..... ] = .....
    return dicoPredecesseurs
```

L'exécution de la fonction `predecesseurs(CAMP_GSMG_LSTADJ, "BUJF")` produit le résultat suivant :

```
{'BUJF': 'BUJF', 'PISC': 'BUJF', 'STAP': 'BUJF', 'BIOL': 'BUJF', 'BUDL': 'BUJF', 'DIDE': 'BUJF', 'DLST': 'BUJF', 'MUSE': 'PISC', 'COND': 'BUDL', 'PMFR': 'BUDL', 'BERL': 'DLST', 'TAIL': 'MUSE'}
```

En exploitant ce dictionnaire des prédécesseurs, déterminer les 2 chemins suivants :

- BUJF à TAIL :
- BUJF à BERL :

La fonction `chemin(depart, fin, graphe)` ci-dessous permet de reconstituer le chemin entre un sommet `depart` et le sommet `fin`.

Compléter les instructions manquantes de la fonction.

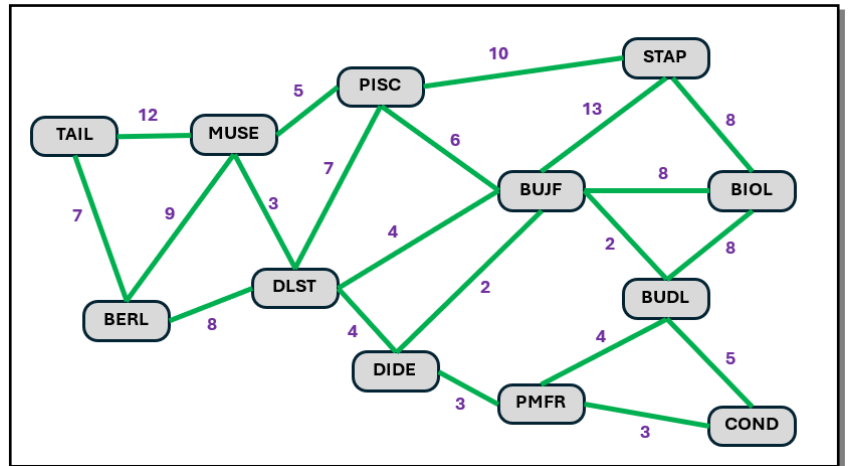
```
def chemin(depart, fin, graphe):
    dicoPredecesseurs = predecesseurs(graphe, depart)
    chemin = [ ..... ]
    predecesseur = .....
    while predecesseur != .....:
        predecesseur = dicoPredecesseurs[ ..... ]
        chemin = [ ..... ] + .....
    return chemin
```

2/ ETUDE AVEC PRISE EN COMPTE DES PONDERATIONS

2.1/LISTE & MATRICE D'ADJACENCE

Représenter le graphe sous la forme :

- D'une matrice d'adjacence pondérée
- D'une liste d'adjacence pondérée



BUJF | PISC | STAP | BIOL | BUDL | DIDE | DLST | MUSE | COND | PMFR | BERL | TAIL

```
[ [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # BUJF
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # PISC
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # STAP
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # BIOL
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # BUDL
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # DIDE
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # DLST
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # MUSE
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # COND
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # PMFR
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ], # BERL
  [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ... ] # TAIL
```

TD-Parcours d'un graphe-Campus GSMH.docx